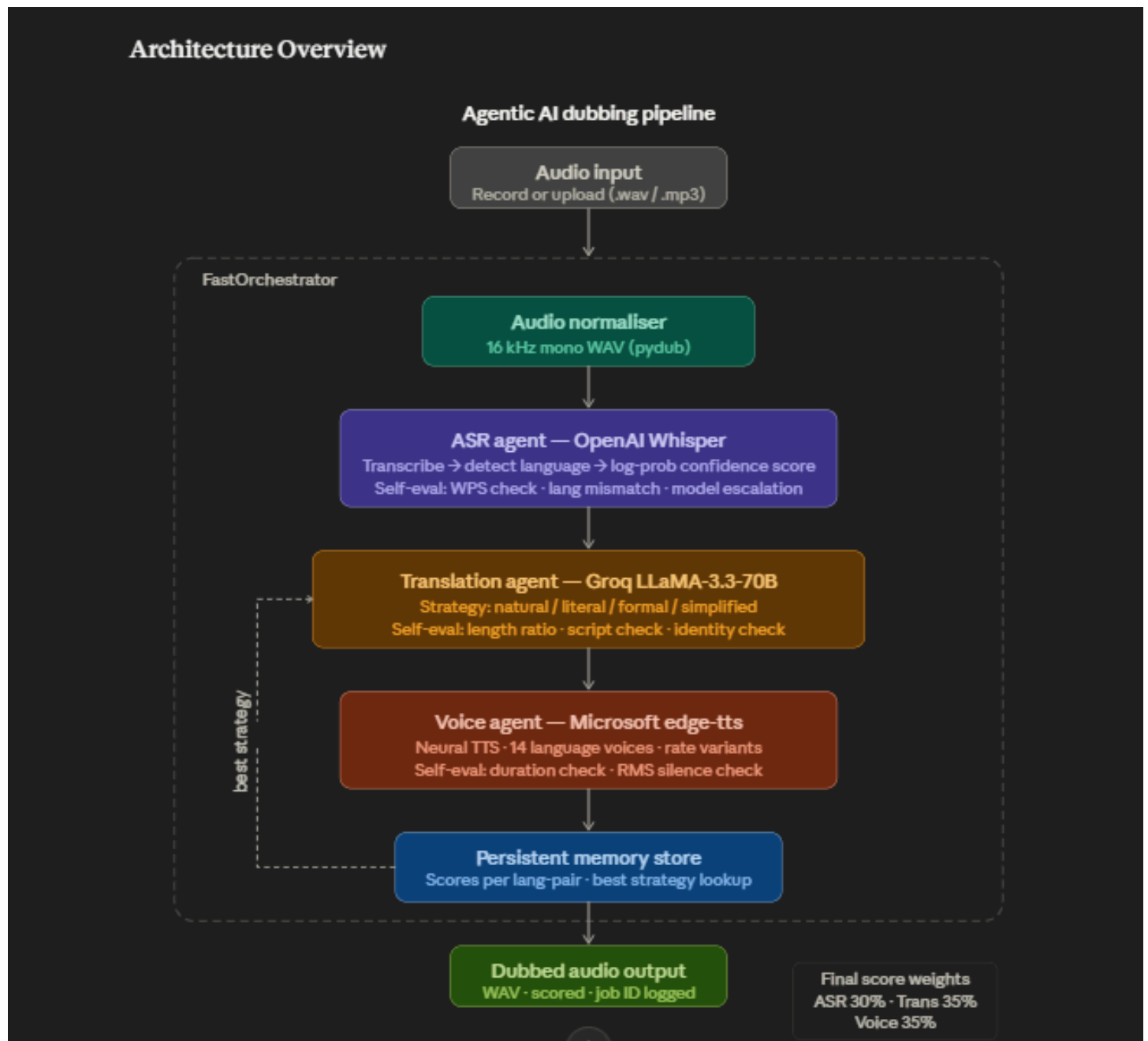


Agentic AI Dubbing System – Detailed Technical Breakdown



Problem Statement (India-Specific Context)

In India, regional movies (e.g., a Malayalam film dubbed into Hindi or Tamil into Kannada) suffer from:

- **Lip-sync mismatch** — dubbed voice doesn't match mouth movements
- **Emotional disconnect** — robotic TTS voice vs. original actor's emotional delivery
- **Prosody failure** — wrong speech rhythm, stress, and intonation
- **Identity loss** — viewer can't connect with the dubbed voice

Your project addresses the **core pipeline** of this problem. Let me break down every layer.

Architecture Overview---

The 3 Specialist Agents – In Depth

Agent 1 – ASR Agent (OpenAI Whisper)

Model used: openai-whisper – specifically the tiny model at runtime (with escalation to small → medium → large if confidence is low).

What it does:

- Takes the normalized 16kHz mono WAV and runs `whisper_model.transcribe()`
- Returns the transcript, detected language, and most importantly the **average log-probability** across all segments (`avg_logprob`)
- `avg_logprob` is Whisper's internal confidence measure – closer to 0 is confident, below -1.5 means the model is essentially guessing

Self-evaluation logic (the `evaluate()` method):

Check	Penalty	Reason
<code>logprob < -1.5</code>	-0.50	Very low ASR confidence
<code>logprob < -0.8</code>	-0.25	Moderate uncertainty
Language mismatch	-0.20	Expected kn, got hi
<code>Words-per-second < 0.3</code>	-0.15	Transcript too sparse for audio length
<code>WPS > 12</code>	-0.10	Hallucination indicator
<code>Transcript < 2 words</code>	-0.30	Basically empty

Model escalation: If ASR score is below `ASR_CONF_THRESHOLD = 0.55`, the orchestrator can call `escalate_model()` which steps up: tiny → base → small → medium → large. This costs time but recovers accuracy for difficult audio like fast speech or heavy accents – critical for Indian regional languages.

Agent 2 – Translation Agent (Groq LLaMA-3.3-70B)

Model used: llama-3.3-70b-versatile via Groq API — chosen because Groq's inference hardware (LPUs) makes 70B-class model calls very fast (often under 1 second).

4 strategy prompts — why they matter for Indian cinema:

- natural — Best for dialogue; preserves idiom, tone, and culturally-loaded expressions. A character saying "ಏನ್ ಮಾಡ್ತೀಯ್?" in Kannada should become "Kya kar raha hai?" in Hindi, not a literal grammatical translation.
- literal — Useful for news, announcements, or when meaning must not shift
- formal — Period dramas, official speeches, courtroom scenes
- simplified — Children's films, comedy dubbed for mass audiences

Self-evaluation logic:

Check	Penalty
Translation empty	Score = 0
Word ratio < 0.25× source	-0.40 (too short, likely truncated)
Word ratio > 5× source	-0.25 (added hallucinated content)
Non-Latin chars in Latin-target lang	-0.40 (source language leaked through)
Translation identical to source	-0.50 (model failed completely)

Translation caching: Uses a translation_cache dict keyed by src_text + src_lang + tgt_lang. Repeated runs of the same phrase skip the LLM entirely — critical for scenes with repeated dialogue or title cards.

Agent 3 — Voice Agent (Microsoft edge-tts)

Model used: Microsoft's Neural TTS via the edge-tts Python library — free, high-quality, works without a key.

14 language voices mapped:

Code	Voice
kn	kn-IN-SapnaNeural
hi	hi-IN-SwaraNeural
ta	ta-IN-PallaviNeural
te	te-IN-ShrutiNeural
en	en-US-JennyNeural
+ 9 more	European + Asian

Speaking rate variants are used on retry: +0% → -10% → +10% → -5%. This matters for lip sync — a slower rate can sometimes align better with the original actor's mouth timing.

Self-evaluation logic:

Check	Penalty
File doesn't exist	Score = 0
Audio duration < 0.4s	-0.50
RMS < 0.002 (silence)	-0.50
RMS < 0.006 (very quiet)	-0.20

TTS caching: Uses @lru_cache keyed by (text, lang_code, rate) — the WAV file is generated once and copied for all subsequent uses.

The Orchestrator (FastOrchestrator)

This is the **coordinator agent** that runs the full pipeline and makes retry decisions.

Input audio

- normalize (pydub, 16kHz mono)
- Whisper transcribe
- memory.best_strategy(src, tgt) ← learns from past jobs
- Groq translate (with cache)
- edge_speak_fast (with cache)
- score all 3 stages
- memory.record_job()
- return result dict

The orchestrator uses **memory-guided strategy selection** — before calling the translation agent, it looks up memory.best_strategy(src_lang, tgt_lang). If the system has dubbed Kannada→Hindi before, it already knows which of the 4 strategies scored highest and uses that first, skipping trial-and-error.

Persistent Memory Store (AgentMemory)

This is what makes the system truly **agentic** — it learns across sessions.

Stored as agent_memory.json:

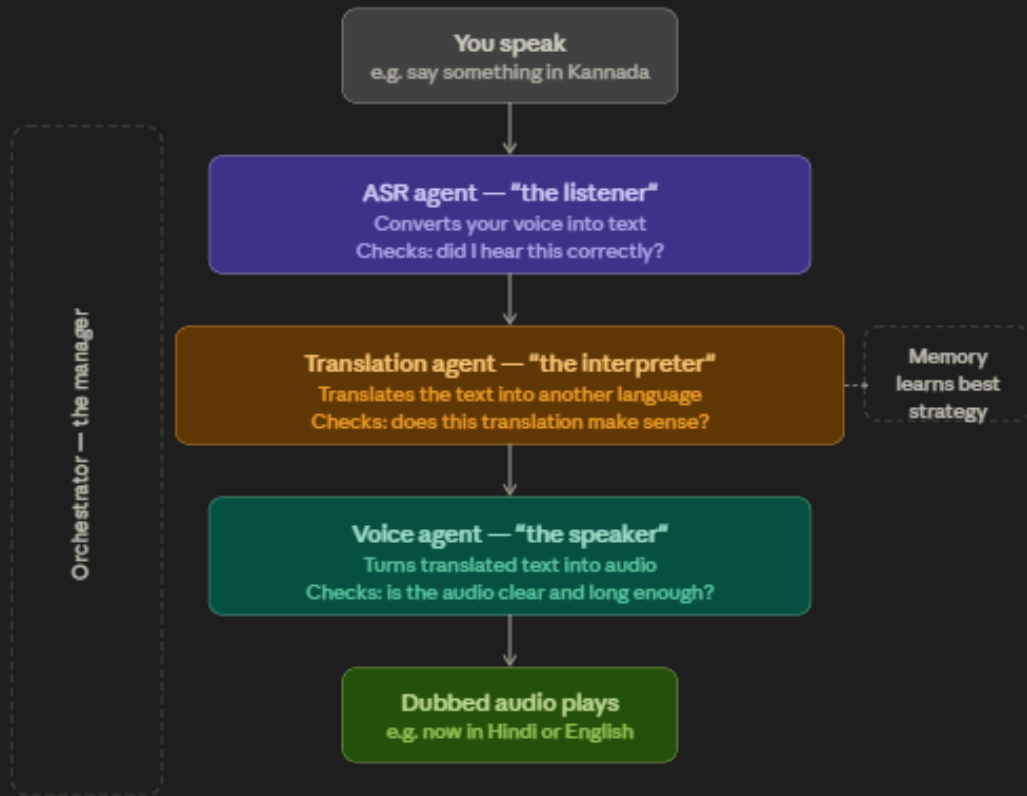
```
{  
  "jobs": [...],  
  "strategy_scores": {  
    "kn->hi": { "natural": [0.82, 0.79, 0.85], "formal": [0.71] }  
  },  
  "total_jobs": 14,  
  "total_success": 13  
}
```

best_strategy() averages scores per strategy per language pair and returns the winner. Over time, the system learns that natural works best for Kannada→Hindi conversational dialogue while formal might score better for Telugu→English documentary content.

How the project works — simply explained

Think of it like a **dubbing studio with 4 workers and a manager**.

You speak in Kannada → the system listens, understands, translates, and speaks back in another language.



Here's the simple breakdown of each agent:

Is agentic AI actually required here? Yes — and here is the honest reason why

A simple non-agentic script would do this: record → transcribe → translate → speak. Done in 20 lines of code. No agents needed.

So why is agentic AI genuinely necessary for the *real* dubbing problem?

Because each step can silently fail, and a dumb pipeline never knows it.

Imagine you record a noisy Kannada sentence. A plain script transcribes it wrong, translates the wrong text perfectly, and speaks out a fluent but completely wrong Hindi sentence. It has no idea it failed. The viewer hears nonsense. This is exactly the bad dubbing problem you started with.

Agentic AI is what makes the difference:

The ASR agent doesn't just transcribe — it checks its own confidence score. If it got a bad result (low log-probability, wrong language detected, too few words), it *knows* it failed and escalates to a better Whisper model automatically. A plain script cannot do this.

The translation agent doesn't just translate — it checks if the output is too short, too long, or still contains the source language characters. If something looks wrong, the orchestrator can retry with a different strategy. A plain script cannot do this.

The memory system means the orchestrator gets smarter with every job. It learns that for Kannada → Hindi, the "natural" strategy consistently scores highest, so it stops wasting time on other strategies. A plain script has no memory and treats every job as its first.

The orchestrator coordinates all three agents, collects their scores, and decides whether the overall result is good enough. This is a *decision-making loop*, not a linear script.

So the honest answer is: without agentic AI, you get a dubbing script that sometimes works and often silently fails with no way to recover. With agentic AI, you get a system that catches its own failures, retries intelligently, and improves over time — which is exactly what the real Indian regional dubbing problem demands.